

PROJETO 6

BIBLIOTECAS E PRIMEIRO CONTROLE DE SERVO

Visual Studio Code + PlatformIO + Wokwi + Arduino UNO

MISSÃO Entender de onde vêm as bibliotecas, aprender a adicioná-las ao projeto e controlar um servomotor em posições definidas.

Curso Técnico em Desenvolvimento de Sistemas
Fundamentos de Eletroeletrônica Aplicada
Manual do aluno • Setembro de 2026

Apresentação

Até aqui você já utilizou comandos do Arduino e componentes simulados. Nesta aula aparece uma nova necessidade: controlar um servomotor. Em vez de receber uma linha pronta para copiar, você aprenderá o caminho completo: identificar a necessidade, procurar uma biblioteca, verificar sua origem, declará-la no projeto, incluí-la no código e consultar os comandos que ela oferece.

ROTA DA AULA necessidade → biblioteca → PlatformIO Registry → platformio.ini → #include → documentação → circuito → código → Build → Wokwi → observação

1. O que é uma biblioteca?

Uma biblioteca é um conjunto reutilizável de código que acrescenta recursos ao programa. Ela evita que cada projeto precise implementar do zero toda a lógica de controle de um dispositivo. Isso não significa usar código sem compreender: primeiro identificamos o que precisamos, depois verificamos a biblioteca e consultamos sua documentação.

Comandos que você já encontrou no ambiente Arduino

```
pinMode()  
digitalWrite()  
analogRead()  
delay()  
tone()
```

Para controlar um servo de forma simples, usaremos a biblioteca Servo. A organização Arduino mantém essa biblioteca para controle de servomotores do tipo hobby. A documentação apresenta, entre outros recursos, `attach()` para associar o objeto a um pino e `write()` para definir a posição.

Do problema à biblioteca usada no código

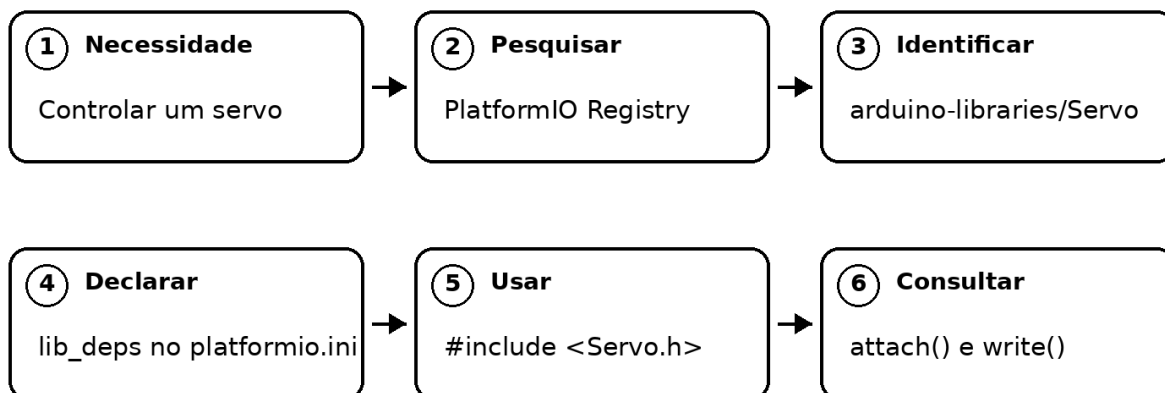


Figura 1 — Caminho que será seguido antes de escrever o primeiro programa.

2. Onde procurar uma biblioteca?

O PlatformIO possui um Registry: um catálogo de pacotes e bibliotecas. A pesquisa pode ser feita no catálogo do PlatformIO. Na interface visual do PIO Home, quando a seção Libraries estiver disponível, abra Libraries, permaneça na guia Registry e use a caixa Search libraries. Se a sua versão do PIO Home não mostrar essa tela, use o Registry pelo navegador ou a pesquisa por pacote no terminal. O objetivo é o mesmo: localizar a biblioteca pelo recurso desejado e verificar sua origem antes de adicioná-la ao projeto.

PESQUISA DA AULA Digite Servo. Não escolha apenas pelo nome: observe proprietário, descrição, compatibilidade, documentação, exemplos e versão.

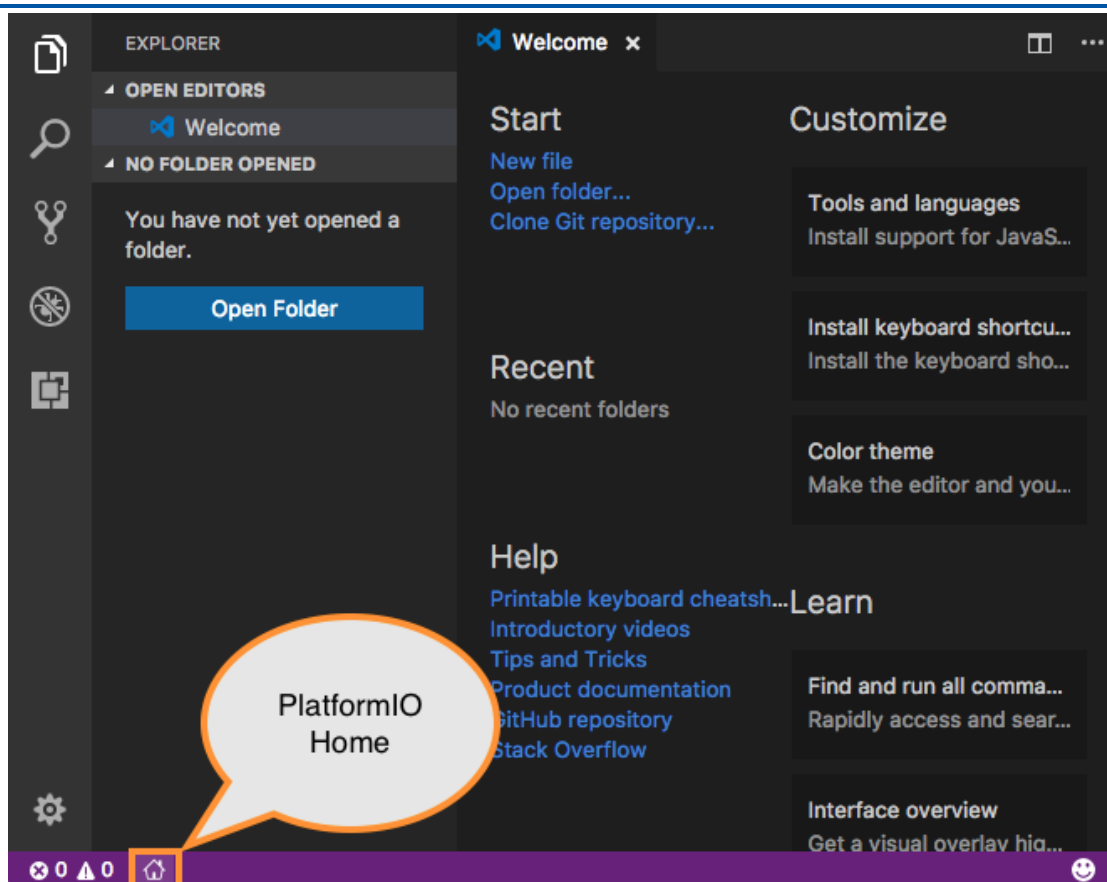


Figura 2 — PlatformIO Home no VS Code. A aparência pode variar conforme a versão instalada.

No nosso caso, a biblioteca escolhida é identificada como `arduino-libraries/Servo`. O prefixo identifica o proprietário/organização e `Servo` é o nome do pacote. Usar proprietário + nome reduz ambiguidades quando existem bibliotecas com nomes semelhantes.

Identificação da biblioteca usada nesta aula

`arduino-libraries/Servo`

3. De onde vem Servo?

A biblioteca `Servo` está no repositório oficial `arduino-libraries/Servo`. A documentação informa que ela permite controlar servomotores RC/hobby e apresenta exemplos como `Sweep`. Para esta aula, os comandos essenciais serão `Servo`, `attach()` e `write()`.

PASSO	FAÇA	CONFIRA
-------	------	---------

1	Pesquise por Servo.	Há vários resultados possíveis.
2	Procure o proprietário Arduino Libraries.	Não confunda com bibliotecas para placas ou arquiteturas específicas.
3	Leia a descrição e compatibilidade.	A biblioteca deve atender ao Arduino UNO/AVR.
4	Localize exemplos/documentação.	Procure attach() e write().

4. O papel do platformio.ini

platformio.ini é o arquivo de configuração do projeto PlatformIO. Nele ficam a plataforma, a placa, o framework e as dependências do projeto. O campo lib_deps declara bibliotecas que o projeto necessita. O PlatformIO resolve e instala essas dependências quando o projeto é processado/compilado.

platformio.ini do Projeto 6

```
[env:uno]
platform = atmelavr
board = uno
framework = arduino

lib_deps =
    arduino-libraries/Servo @ ^1.3.0
```

ENTENDA lib_deps declara a dependência do projeto. Já #include <Servo.h> informa ao código-fonte que ele usará o cabeçalho da biblioteca. São etapas relacionadas, mas não são a mesma coisa.

Do projeto para o código

```
platformio.ini
lib_deps = arduino-libraries/Servo
```

↓
PlatformIO resolve a dependência
↓
src/main.cpp
#include <Servo.h>

5. Criando o projeto no PlatformIO

Abra o PIO Home e escolha New Project. Dê um nome ao projeto, selecione Arduino Uno como placa e Arduino como framework. O nome do ambiente criado será uno quando o platformio.ini usar [env:uno].

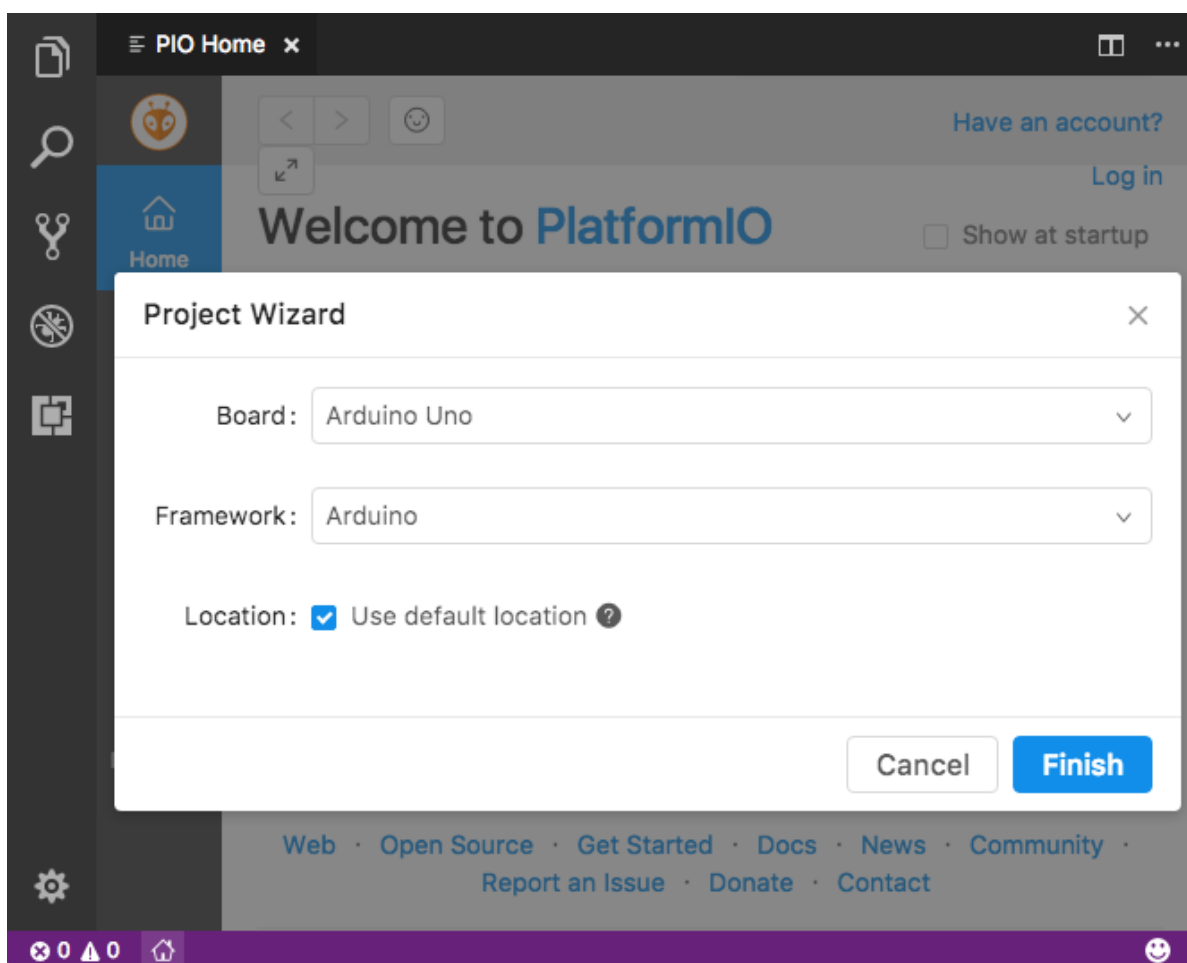


Figura 3 — Project Wizard: Arduino Uno e framework Arduino.

PASSO

FAÇA

CONFIRA

1	Abra PIO Home.	A página inicial do PlatformIO aparece.
2	Escolha New Project.	O Project Wizard é aberto.
3	Nomeie como Projeto_6_Servo.	Use uma pasta que você consiga localizar.
4	Board: Arduino Uno.	Confirme a placa.
5	Framework: Arduino.	Confirme o framework e finalize.

6. Estrutura que vamos usar

O código-fonte fica em `src/main.cpp`. Na raiz ficam `platformio.ini`, `diagram.json` e `wokwi.toml`. Após o Build, o PlatformIO cria `.pio/build/uno` e gera os arquivos que o Wokwi executará.

Estrutura do Projeto 6 no VS Code



Figura 4 — Arquivos do Projeto 6 e a função de cada um.

7. Preparando o Wokwi

Para a simulação local no VS Code, o Wokwi precisa saber qual firmware executar e qual circuito representar. Por isso usamos dois arquivos na raiz: wokwi.toml e diagram.json.

wokwi.toml

```
[wokwi]
version = 1
firmware = '.pio/build/uno/firmware.hex'
elf = '.pio/build/uno/firmware.elf'
```

firmware aponta para o arquivo HEX gerado pelo Build. elf aponta para o arquivo ELF. Os caminhos são relativos à raiz do projeto e dependem do nome do ambiente [env:uno]. Use barras / nos caminhos.

8. Primeiro circuito: Arduino UNO + servo

Um servo hobby possui três conexões: alimentação, terra e sinal de controle. Nesta simulação usaremos 5V, GND e o pino digital D5 para o sinal. Em montagem física, servos podem exigir fonte separada quando a corrente aumenta; nesta prática introdutória o objetivo é compreender o controle no simulador.

Como o servo entra no sistema



Energia: 5V e GND | Controle: sinal digital no pino escolhido

Figura 5 — O programa controla a posição do eixo por meio do sinal no pino D5.

diagram.json — Arduino UNO e um servo

```
{
  "version": 1,
  "author": "Hilton Elias",
  "editor": "wokwi",
  "parts": [
    {
      "type": "wokwi-arduino-uno",
      "id": "uno",
      "top": 0,
      "left": 0,
      "attrs": {}
    },
    {
      "type": "wokwi-servo",
      "id": "servo1",
      "top": -130,
      "left": 300,
      "attrs": {}
    }
  ],
  "connections": [
    ["servo1:PWM", "uno:5", "orange", []],
    ["servo1:V+", "uno:5V", "red", []],
    ["servo1:GND", "uno:GND.1", "black", []]
  ],
  "dependencies": {}
}
```

PASSO	FAÇA	CONFIRA
V+	Ligue em 5V.	Alimentação do servo.
GND	Ligue em GND.	Referência elétrica comum.
PWM/Sinal	Ligue em D5.	Pino usado pelo objeto Servo.

9. Como a biblioteca aparece no código

Primeiras linhas de src/main.cpp

```
#include <Arduino.h>
#include <Servo.h>
```

```
Servo s1;
```

Servo s1; cria um objeto chamado s1. No setup(), s1.attach(5) associa esse objeto ao pino D5. Depois, s1.write(angulo) solicita uma posição. Para um servo padrão, trabalharemos didaticamente com a faixa de 0° a 180°.

Três instruções fundamentais

```
s1.attach(5);    // associa o servo ao pino 5
s1.write(0);     // posição inicial
s1.write(90);    // posição central
s1.write(180);   // posição final
```

10. Primeiro programa

src/main.cpp

```
#include <Arduino.h>
#include <Servo.h>

// Cria um objeto para controlar o servo
Servo s1;

void setup() {
    // Liga o objeto s1 ao pino digital 5
    s1.attach(5);
}

void loop() {
    // Vai para 0 graus
    s1.write(0);
    delay(1000);

    // Vai para 90 graus
    s1.write(90);
    delay(1000);
}
```

```
// Vai para 180 graus
s1.write(180);
delay(1000);

// Retorna para 90 graus
s1.write(90);
delay(1000);
}
```

11. Compilar antes de simular

Salve os arquivos e execute Build. O Build verifica o código, resolve dependências declaradas e gera o firmware. Só avance quando o terminal indicar SUCCESS. Depois do Build, confirme a existência de .pio/build/uno/firmware.hex e firmware.elf.

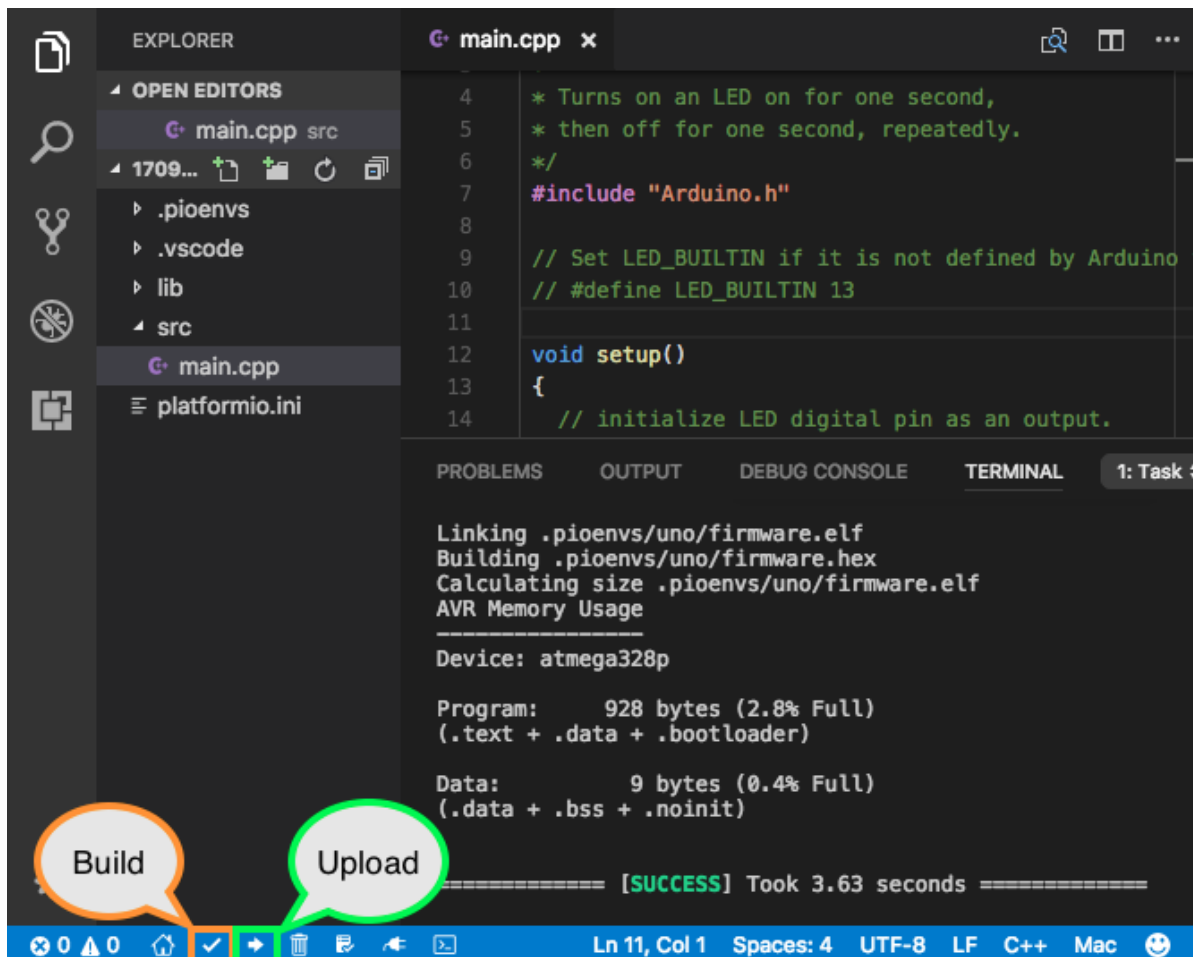


Figura 6 — Exemplo de Build concluído com SUCCESS no PlatformIO.

12. Executar no Wokwi

Com o firmware atualizado, inicie o simulador Wokwi no VS Code. Se usar a Paleta de Comandos, procure Wokwi: Start Simulator. O Wokwi carrega o firmware indicado em wokwi.toml e o circuito descrito em diagram.json.

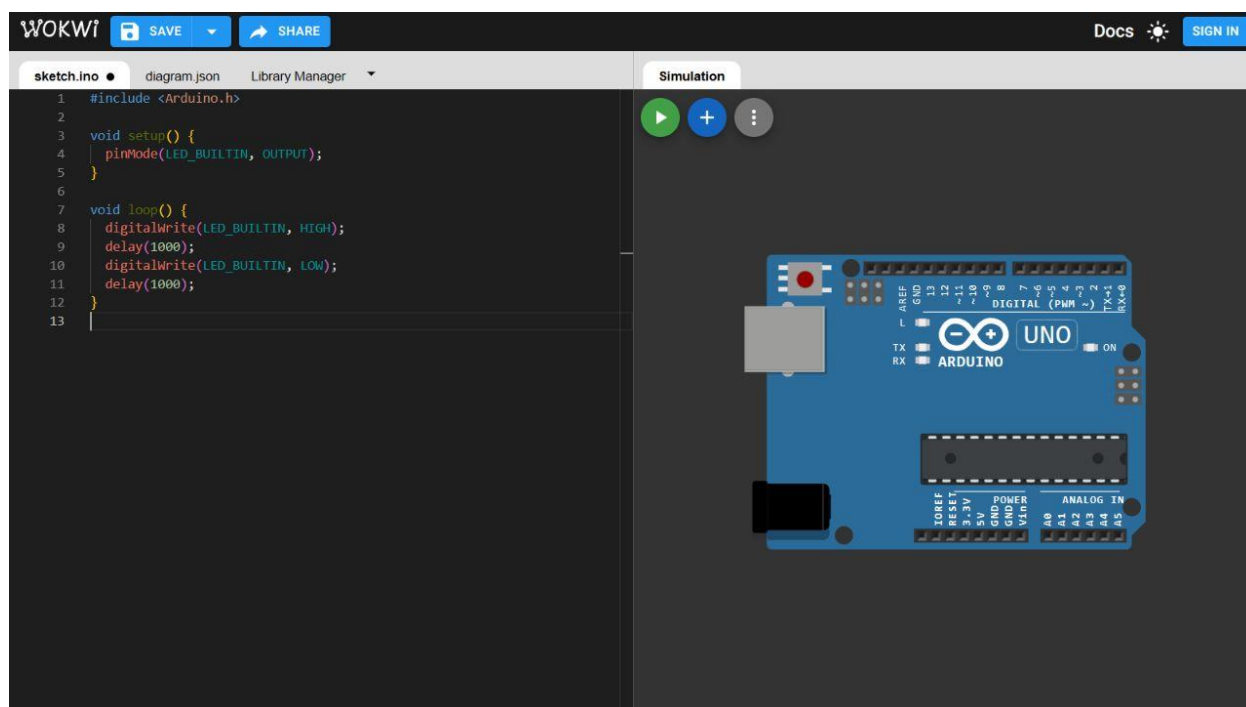


Figura 7 — Exemplo de interface do Wokwi. O Projeto 6 terá um servo conectado ao Arduino UNO.

RESULTADO ESPERADO O servo muda de posição aproximadamente a cada 1 segundo: 0° → 90° → 180° → 90° e reinicia o ciclo.

13. Prática guiada: prever, modificar e observar

MÉTODO MODIFIQUE → PREVEJA → COMPILE → EXECUTE → OBSERVE → COMPARE → CONCLUA

Antes de executar cada alteração, registre o que você acredita que acontecerá. Depois faça Build, execute a simulação e compare a observação com sua previsão.

TESTE	ALTERAÇÃO	PREVISÃO	OBSERVAÇÃO
A	write(45)	_____	_____
B	write(135)	_____	_____
C	delay(500)	_____	_____
D	ordem 180 → 90 → 0	_____	_____

14. Movimento gradual

write() define uma posição. Para criar uma sequência de pequenas mudanças, podemos combinar write() com for. O valor de ang aumenta de 0 até 180 e cada valor é enviado ao servo.

Movimento gradual de 0° até 180°

```
for (int ang = 0; ang <= 180; ang++) {
    s1.write(ang);
    delay(15);
}
```

Compare com s1.write(180): no primeiro caso o programa envia muitas posições intermediárias; no segundo, solicita diretamente a posição final.

15. Exercício antes da atividade final

Crie um ciclo que faça o servo: 0° → 45° → 90° → 135° → 180° → 135° → 90° → 45° → 0°. Escolha uma temporização que permita observar cada posição. Depois substitua as posições fixas por dois laços for, um crescente e outro decrescente.

16. Diagnóstico rápido

PASSO	FAÇA	CONFIRA
Servo.h não encontrado	Dependência não resolvida.	Confira lib_deps e execute Build novamente.
Build falha	Erro de sintaxe ou arquivo não salvo.	Leia a primeira linha de erro e confira main.cpp.

Wokwi não inicia	Firmware/caminho/extensão.	Confirme SUCCESS e wokwi.toml.
Servo não move	Ligação ou pino diferente.	Confira diagram.json e attach(5).
Mudança não aparece	Firmware antigo.	Salve, faça Build e reinicie a simulação.

17. Checklist

- ☐ Entendi o que é uma biblioteca.
- ☐ Sei onde pesquisar uma biblioteca no ecossistema PlatformIO.
- ☐ Identifiquei arduino-libraries/Servo e consultei sua documentação.
- ☐ Entendi a diferença entre lib_deps e #include.
- ☐ Criei/abri o projeto para Arduino Uno.
- ☐ Configurei platformio.ini.
- ☐ Criei diagram.json e wokwi.toml.
- ☐ Liguei o servo em 5V, GND e D5.
- ☐ Usei Servo, attach() e write().
- ☐ Compilei com SUCCESS.
- ☐ Executei no Wokwi e observei as posições.
- ☐ Testei movimento gradual com for.

18. Preparação para a atividade final

Na atividade final, você receberá um sistema maior com sensor ultrassônico, dois servos, LEDs e buzzer. O objetivo será integrar conhecimentos já estudados. O código completo da porta automática não é fornecido neste manual do aluno.

VOCÊ ESTÁ PRONTO QUANDO consegue explicar de onde veio Servo.h, configurar a dependência, controlar um servo e interpretar attach() e write() sem apenas copiar o código.

Referências consultadas

- PlatformIO. Dependency Management e Library Registry.
- PlatformIO. lib_deps — Project Configuration.
- PlatformIO IDE for VSCode — criação de projeto.
- Wokwi Docs. Configuring Your Project (wokwi.toml).
- Arduino Libraries. Servo Library for Arduino — documentação e API.